

1. **SQL languages:** The sub-languages of SQL · DDL: defining the schema · DML: reading and changing rows · DCL: users, roles and privileges · TCL: transactions
2. **Constraints:** PRIMARY KEY, UNIQUE and NOT NULL · FOREIGN KEY and referential actions · CHECK and DEFAULT
3. **Indexes:** What an index is · Composite and covering indexes · What indexes cost · Kinds of index
4. **Clustering:** Clustered and non-clustered indexes · Physical order in PostgreSQL and Oracle
5. **Partitioning:** Horizontal partitioning · Partition pruning and maintenance · Vertical partitioning and sharding
6. **Efficiency:** Reading a query plan · Conditions that can use an index · Writing efficient queries

SQL languages

The sub-languages of SQL

SQL is one language with several **sub-languages**, grouped by what a statement does: define the structure, work with the data, control who can do what, and group changes into transactions.

The four groups

Sub-language	Statements	What it acts on
DDL · data definition	CREATE, ALTER, DROP, TRUNCATE, RENAME	The schema: tables, columns, constraints, indexes, views
DML · data manipulation	SELECT, INSERT, UPDATE, DELETE, MERGE	The rows stored in the tables
DCL · data control	GRANT, REVOKE	Users, roles and their privileges
TCL · transaction control	BEGIN, COMMIT, ROLLBACK, SAVEPOINT	When changes become permanent or are undone

Example. CREATE TABLE student (...) is DDL; INSERT INTO student VALUES (...) is DML; GRANT SELECT ON student TO teacher is DCL; COMMIT is TCL.

Common mistake. Thinking TRUNCATE is the same as DELETE without WHERE. TRUNCATE is DDL: it empties the table at once, usually cannot be filtered, and in many DBMSs commits implicitly.

DDL: defining the schema

CREATE builds an object, **ALTER** changes it and **DROP** removes it, data included. DDL turns the logical model into real tables.

- Types: INTEGER, NUMERIC(p, s), VARCHAR(n) / TEXT, DATE, TIMESTAMP, BOOLEAN... (each DBMS has its own list).
- Each DDL statement updates the data dictionary, which you can query: here, `sqlite_master`.

Example you can run on the website

```
CREATE TABLE course (
  course_id TEXT PRIMARY KEY,
  title     TEXT NOT NULL,
  credits   REAL NOT NULL
);
ALTER TABLE course ADD COLUMN semester INTEGER;
INSERT INTO course VALUES ('C10', 'Databases', 6, 1);
SELECT * FROM course;
SELECT name, sql FROM sqlite_master WHERE type = 'table';
DROP TABLE course;
SELECT name FROM sqlite_master WHERE type = 'table';
```

Common mistake. Dropping and re-creating a table to change one column. ALTER TABLE keeps the rows; DROP loses them.

DML: reading and changing rows

SELECT reads, **INSERT** adds rows, **UPDATE** changes them and **DELETE** removes them. UPDATE and DELETE

affect every row that matches the WHERE.

- Joins follow the foreign keys: enrolment.course_id = course.course_id.
- The runner shows how many rows each INSERT, UPDATE or DELETE changed.

Example you can run on the website

```
INSERT INTO enrolment VALUES ('S03', 'C20', 6.0);
UPDATE enrolment SET grade = grade + 0.5 WHERE course_id = 'C10' AND grade < 6;
DELETE FROM enrolment WHERE student_id = 'S02' AND course_id = 'C11';
SELECT s.name, c.title, e.grade
FROM enrolment e
JOIN student s ON s.student_id = e.student_id
JOIN course c ON c.course_id = e.course_id
ORDER BY s.name, c.title;
SELECT c.title, COUNT(*) AS students, ROUND(AVG(e.grade), 2) AS average
FROM enrolment e JOIN course c ON c.course_id = e.course_id
GROUP BY c.title
HAVING COUNT(*) >= 2;
```

Example. The average grade per course is `SELECT course_id, AVG(grade) FROM enrolment GROUP BY course_id`.

Common mistake. Filtering an aggregate with WHERE (`WHERE AVG(grade) > 7`). WHERE runs before grouping; use `HAVING AVG(grade) > 7`.

DCL: users, roles and privileges

GRANT gives a privilege on an object to a user or role; **REVOKE** takes it away. Give each role only what it needs: **the least privilege principle**.

- WITH GRANT OPTION lets the receiver grant the same privilege to others; use it sparingly.
- REVOKE removes the privilege from that role; members lose it unless another role still grants it.

PostgreSQL · not runnable here

```
CREATE ROLE teacher;
CREATE ROLE reporting;
CREATE USER ana PASSWORD 'change-me' IN ROLE teacher;

GRANT SELECT, INSERT, UPDATE ON enrolment TO teacher;
GRANT SELECT ON student, course TO teacher;

CREATE VIEW course_average AS
  SELECT course_id, AVG(grade) AS average FROM enrolment GROUP BY course_id;
GRANT SELECT ON course_average TO reporting;      -- no access to individual grades

REVOKE UPDATE ON enrolment FROM teacher;
```

Common mistake. Letting the application connect as the database owner. A bug or an SQL injection can then drop tables; with a role limited to DML on its tables, it cannot.

TCL: transactions

A **transaction** groups statements that must succeed or fail together. **COMMIT** makes them permanent, **ROLLBACK** undoes them, and a **SAVEPOINT** marks a point you can roll back to.

- `SAVEPOINT name ... ROLLBACK TO name` undoes only the work after the savepoint.
- Keep transactions short: while one is open, it may hold locks that make others wait.

Example you can run on the website

```
BEGIN;
DELETE FROM enrolment WHERE student_id = 'S03' AND course_id = 'C11';
INSERT INTO enrolment VALUES ('S03', 'C20', NULL);
SAVEPOINT before_grades;
UPDATE enrolment SET grade = 10;          -- oops: every row
ROLLBACK TO before_grades;
```

```
COMMIT;
SELECT * FROM enrolment WHERE student_id = 'S03';
BEGIN;
DELETE FROM enrolment;
SELECT COUNT(*) AS rows_inside_transaction FROM enrolment;
ROLLBACK;
SELECT COUNT(*) AS rows_after_rollback FROM enrolment;
```

Example. The second block changes a grade by mistake and rolls back to the savepoint: the move between courses is kept, the wrong grade is not.

Common mistake. Running a long script in autocommit mode. If it fails halfway, the first half stays applied and the database is left in a state nobody planned.

Constraints

PRIMARY KEY, UNIQUE and NOT NULL

The **primary key** identifies each row: unique and never **NULL**. **UNIQUE** forbids duplicates in other candidate keys, and **NOT NULL** makes a column mandatory.

- Composite key: PRIMARY KEY (student_id, course_id).
- The last statement of the example fails on purpose: read the error message.

[Example you can run on the website](#)

```
INSERT INTO student VALUES ('S04', 'Marta Paz', NULL); -- a second NULL e-mail is allowed
SELECT * FROM student;
INSERT INTO student VALUES ('S05', 'Ana Ruiz', 'ana@uni.es'); -- duplicate e-mail
```

Common mistake. Using only a surrogate id and forgetting the natural key. With id as PK and no UNIQUE (email), the same student can be inserted twice.

FOREIGN KEY and referential actions

A **foreign key** only accepts values that exist in the referenced key. Its **referential action** says what happens to child rows when the parent row is deleted or its key changes.

- CASCADE fits parts that make no sense alone (an enrolment without its student, an order line without its order).
- RESTRICT protects reference data (a course, a department).
- SQLite only checks foreign keys after PRAGMA foreign_keys = ON; the runner turns it on.

[Example you can run on the website](#)

```
DELETE FROM student WHERE student_id = 'S01'; -- CASCADE: Ana's enrolments go too
SELECT * FROM enrolment ORDER BY student_id;
DELETE FROM course WHERE course_id = 'C10'; -- C10 still has enrolments: rejected
```

Common mistake. Using CASCADE everywhere. Deleting one department could then silently delete its courses and all their enrolments.

CHECK and DEFAULT

CHECK states a condition each row must meet, such as a grade between 0 and 10. **DEFAULT** gives a column its value when the INSERT does not say one.

- A CHECK passes when the condition is true or unknown (NULL); add NOT NULL if the value is required.

[Example you can run on the website](#)

```
CREATE TABLE loan (
  loan_id  INTEGER PRIMARY KEY,
  book     TEXT NOT NULL,
  loan_date TEXT NOT NULL DEFAULT (date('now')),
  days     INTEGER NOT NULL DEFAULT 15,
```

```

status TEXT NOT NULL DEFAULT 'open',
CONSTRAINT days_range CHECK (days BETWEEN 1 AND 60),
CONSTRAINT status_values CHECK (status IN ('open', 'returned', 'lost'))
);
INSERT INTO loan (book) VALUES ('Don Quixote');
INSERT INTO loan (book, days, status) VALUES ('Ficciones', 30, 'returned');
SELECT * FROM loan;
INSERT INTO loan (book, days) VALUES ('Hopscotch', 90);      -- breaks days_range

```

Common mistake. Checking ranges only in the application. Data loaded with a script or another program then skips the rule.

Indexes

What an index is

An **index** is a separate, sorted structure (usually a **B+ tree**) that maps the values of some columns to their rows. With it, the DBMS can **search** instead of **scanning** the whole table.

- The primary key and each UNIQUE constraint already have an index.
- Foreign keys usually do **not** get one automatically; index them if you join or filter by them.

Example you can run on the website

```

EXPLAIN QUERY PLAN SELECT * FROM customer WHERE email = 'user43210@mail.com';
SELECT * FROM customer WHERE email = 'user43210@mail.com';
CREATE INDEX idx_customer_email ON customer (email);
EXPLAIN QUERY PLAN SELECT * FROM customer WHERE email = 'user43210@mail.com';
SELECT * FROM customer WHERE email = 'user43210@mail.com';

```

Common mistake. Thinking an index changes the result. It only changes how fast the DBMS finds the rows; the query returns exactly the same data.

Composite and covering indexes

A **composite** index sorts by several columns, in order. It helps queries that filter on its **leftmost** columns. A **covering** index contains every column a query needs, so the table is not read at all.

- Leftmost-prefix rule: (a, b, c) serves filters on a, a, b and a, b, c.
- One composite index can replace several single-column ones.

Example you can run on the website

```

CREATE INDEX idx_city_date ON customer (city, signup_date);
EXPLAIN QUERY PLAN SELECT * FROM customer WHERE city = 'Madrid' AND signup_date >= '2023-01-01';
EXPLAIN QUERY PLAN SELECT * FROM customer WHERE signup_date >= '2023-01-01';
EXPLAIN QUERY PLAN SELECT city, signup_date FROM customer WHERE city = 'Sevilla';
SELECT COUNT(*) AS madrid_since_2023 FROM customer WHERE city = 'Madrid' AND signup_date >= '2023-01-01';

```

Common mistake. Creating (signup_date, city) for queries that always fix the city and give a date range. With the range first, the second column cannot narrow the search.

What indexes cost

Indexes speed up reads but slow down writes: every INSERT, UPDATE or DELETE must also update **each** index of the table. They also take disk and memory.

When to index a column

Usually yes	Usually no
Foreign keys used in joins	Columns never used in WHERE, JOIN or ORDER BY
Columns filtered with = or ranges that return few rows	Low-selectivity columns (gender, yes/no)
Columns used to sort large results (ORDER BY ... LIMIT)	Small tables that fit in a few blocks
Search keys of frequent queries	Tables with heavy writes and rare reads (logs)

Example you can run on the website

```
WITH RECURSIVE n(i) AS (SELECT 1 UNION ALL SELECT i + 1 FROM n WHERE i < 20000)
INSERT INTO plain_log SELECT i, i % 997, 'action' || (i % 13), datetime('2024-01-01', '+' || i || ' minutes') FROM
n;
WITH RECURSIVE n(i) AS (SELECT 1 UNION ALL SELECT i + 1 FROM n WHERE i < 20000)
INSERT INTO indexed_log SELECT i, i % 997, 'action' || (i % 13), datetime('2024-01-01', '+' || i || ' minutes')
FROM n;
SELECT name AS indexes_on_indexed_log FROM sqlite_master WHERE type = 'index' AND tbl_name = 'indexed_log';
```

Common mistake. Indexing every column "just in case". Each index slows every write and the optimizer still uses only the useful ones.

Kinds of index

The **B-tree** is the default and serves equality, ranges and sorting. Other kinds solve specific problems: **hash** for equality only, **bitmap** for few distinct values, **GIN** for values inside a document or array, **BRIN** for huge, naturally ordered tables.

Main index kinds

Kind	Good for	Not for
B-tree / B+ tree	=, <, >, BETWEEN, ORDER BY, prefix LIKE 'abc%'	Searching inside text or arrays
Hash	Equality only (=)	Ranges and sorting
Bitmap	Columns with few distinct values in read-mostly tables (data warehouses)	Tables with many concurrent writes
GIN / inverted	Full-text search, JSON keys, array elements	Simple scalar columns
BRIN	Very large tables ordered by insertion (dates in a log)	Randomly ordered data

PostgreSQL · not runnable here

```
CREATE INDEX orders_customer_idx ON orders (customer_id);           -- B-tree (default)
CREATE INDEX session_token_idx  ON session USING hash (token);     -- equality only
CREATE INDEX product_tags_idx   ON product USING gin (tags);       -- tags is text[]
CREATE INDEX log_at_brin        ON access_log USING brin (at);      -- huge, append-only
CREATE INDEX open_orders_idx    ON orders (order_date) WHERE status = 'open'; -- partial
CREATE INDEX customer_email_ci  ON customer (lower(email));        -- expression
```

Common mistake. Choosing a hash index for a column used in ranges or ORDER BY. It cannot answer either; only a B-tree can.

Clustering

Clustered and non-clustered indexes

In a **clustered** index the table rows themselves are stored in the order of the key, inside the index. A table can have only **one**. Every other index is **non-clustered**: it stores the key plus a pointer to the row.

- Choose a clustering key that is short, does not change, and matches frequent range queries.
- Random keys (such as random UUIDs) scatter inserts all over a clustered index; increasing keys append at the end.

Example you can run on the website

```
CREATE TABLE enrolment_by_course (
  course_id TEXT,
  student_id TEXT,
  grade      REAL,
  PRIMARY KEY (course_id, student_id)
) WITHOUT ROWID;
INSERT INTO enrolment_by_course VALUES
  ('C20', 'S01', 7.0), ('C10', 'S03', 5.0), ('C10', 'S01', 8.5), ('C11', 'S02', 9.0), ('C10', 'S02', 6.5);
SELECT * FROM enrolment_by_course;           -- stored, and returned, in key order
EXPLAIN QUERY PLAN SELECT * FROM enrolment_by_course WHERE course_id = 'C10';
```

Common mistake. Confusing a **clustered index** with a **database cluster**. The second is a group of servers working together (replication, high availability), an unrelated meaning of the word.

Physical order in PostgreSQL and Oracle

PostgreSQL stores tables as **unordered heaps**. Its **CLUSTER** command rewrites a table once in the order of an index, but new rows are not kept in that order. Oracle offers index-organized tables and **table clusters**.

PostgreSQL · not runnable here

```
CREATE INDEX orders_date_idx ON orders (order_date);
CLUSTER orders USING orders_date_idx;      -- rewrites the table in date order (locks it)
ANALYZE orders;                            -- refresh statistics after the rewrite

-- How well the physical order follows a column (1 = perfectly sorted):
SELECT attname, correlation FROM pg_stats
WHERE tablename = 'orders' AND attname = 'order_date';
```

Common mistake. Expecting CLUSTER to keep the table sorted. It is a one-off rewrite; only a true clustered index keeps the order as rows arrive.

Partitioning

Horizontal partitioning

Partitioning splits one big table into smaller pieces, the **partitions**, according to a **partition key**. Queries still use the single table name; the DBMS sends each row to its partition.

Choosing the method

Method	Typical key	Use it when
Range	Date, id ranges	Queries and data retention go by time
List	Country, region, status	A few known values separate the data
Hash	Customer id	You need even sizes and there is no natural range

PostgreSQL · not runnable here

```
CREATE TABLE orders (
  order_id    bigint,
  customer_id bigint NOT NULL,
  order_date  date    NOT NULL,
  total       numeric(10, 2) NOT NULL,
  PRIMARY KEY (order_id, order_date)      -- the partition key must be part of it
) PARTITION BY RANGE (order_date);

CREATE TABLE orders_2025_01 PARTITION OF orders
  FOR VALUES FROM ('2025-01-01') TO ('2025-02-01');
CREATE TABLE orders_2025_02 PARTITION OF orders
  FOR VALUES FROM ('2025-02-01') TO ('2025-03-01');
CREATE TABLE orders_default PARTITION OF orders DEFAULT;  -- rows outside every range

CREATE INDEX ON orders (customer_id);      -- created on every partition
```

Common mistake. Partitioning a small table. Below many millions of rows, a good index is usually simpler and just as fast.

Partition pruning and maintenance

When the **WHERE** fixes the partition key, the optimizer reads only the matching partitions: **partition pruning**. Old data can then be removed by dropping a whole partition instead of deleting millions of rows.

- Choose the partition key from the most frequent filters.
- Unique constraints and the primary key must include the partition key.

PostgreSQL · not runnable here

```
EXPLAIN SELECT SUM(total) FROM orders
WHERE order_date >= '2025-02-01' AND order_date < '2025-03-01';
-- Aggregate
--    -> Seq Scan on orders_2025_02 orders    <- only one partition is read

ALTER TABLE orders DETACH PARTITION orders_2025_01;    -- keep it as a normal table...
DROP TABLE orders_2025_01;                            -- ...or remove a whole month at once
```

Common mistake. Partitioning by a column the queries rarely filter on. Without pruning, every query visits every partition.

Vertical partitioning and sharding

Vertical partitioning splits the **columns** of a table into two tables with the same key. **Sharding** splits the **rows** across different **servers**, each with its own database.

Three ways to split

Technique	Splits	Where the pieces live	Main cost
Horizontal partitioning	Rows	Same database	Queries without the key visit every partition
Vertical partitioning	Columns	Same database	A join to read the whole row
Sharding	Rows	Different servers	Cross-shard joins and transactions

Common mistake. Sharding before it is needed. Indexes, partitioning, read replicas and a bigger server solve most problems with far less complexity.

Efficiency**Reading a query plan**

SQL says **what** you want; the **optimizer** decides **how**: which index, which join order, which algorithm. **EXPLAIN** shows that plan, and the plan is where slow queries are understood.

- SCAN = read every row; SEARCH ... USING INDEX = go down an index.
- USE TEMP B-TREE FOR ORDER BY = an extra sort; an index in that order avoids it.

Example you can run on the website

```
EXPLAIN QUERY PLAN
SELECT c.city, SUM(o.total) FROM customer c JOIN orders o ON o.customer_id = c.customer_id
WHERE c.customer_id BETWEEN 100 AND 200 GROUP BY c.city;
SELECT c.city, ROUND(SUM(o.total), 2) AS spent FROM customer c JOIN orders o ON o.customer_id = c.customer_id
WHERE c.customer_id BETWEEN 100 AND 200 GROUP BY c.city;
CREATE INDEX idx_orders_customer ON orders (customer_id);
ANALYZE;
EXPLAIN QUERY PLAN
SELECT c.city, SUM(o.total) FROM customer c JOIN orders o ON o.customer_id = c.customer_id
WHERE c.customer_id BETWEEN 100 AND 200 GROUP BY c.city;
SELECT c.city, ROUND(SUM(o.total), 2) AS spent FROM customer c JOIN orders o ON o.customer_id = c.customer_id
WHERE c.customer_id BETWEEN 100 AND 200 GROUP BY c.city;
```

Common mistake. Optimizing by intuition. Read the plan first: the slow part is often not where you expect.

Conditions that can use an index

An index on a column only helps when the condition compares the **bare column**. Wrapping it in a function or calculation hides it from the index: the query scans.

Example you can run on the website

```
EXPLAIN QUERY PLAN SELECT COUNT(*) FROM customer WHERE substr(signup_date, 1, 4) = '2023';
EXPLAIN QUERY PLAN SELECT COUNT(*) FROM customer WHERE signup_date >= '2023-01-01' AND signup_date < '2024-01-01';
```

```
EXPLAIN QUERY PLAN SELECT * FROM customer WHERE lower(email) = 'user77@mail.com';  
CREATE INDEX idx_email_lower ON customer (lower(email));  
EXPLAIN QUERY PLAN SELECT * FROM customer WHERE lower(email) = 'user77@mail.com';
```

Common mistake. Adding an index and assuming the query will use it. Check the plan; a function on the column is enough to make the index useless.

Writing efficient queries

Ask the database for exactly what you need, in one set-based statement, and let it do the work near the data: filter, join and aggregate in SQL, not row by row in the application.

- Select only the columns you use; `SELECT *` reads and sends more data and stops covering indexes from helping.
- Use `EXISTS` to ask "is there any?" instead of `COUNT(*) > 0`: it can stop at the first match.
- Paginate with `ORDER BY ... LIMIT`, ideally on an indexed column.
- Join and filter on indexed keys, and keep transactions short.

Example you can run on the website

```
-- "Has customer 42 ordered anything?": EXISTS can stop at the first order  
SELECT EXISTS (SELECT 1 FROM orders WHERE customer_id = 42) AS has_orders;  
-- One set-based statement instead of a loop over every order  
UPDATE orders SET total = round(total * 0.9, 2) WHERE order_date < '2023-02-01';  
-- Only the columns and rows needed, aggregated in the database  
SELECT c.city, COUNT(*) AS orders, ROUND(SUM(o.total), 2) AS spent  
FROM orders o JOIN customer c ON c.customer_id = o.customer_id  
WHERE o.order_date BETWEEN '2024-01-01' AND '2024-12-31'  
GROUP BY c.city  
ORDER BY spent DESC  
LIMIT 5;
```

Example. The last query of the example answers "top 5 cities by spending in 2024" with one statement, instead of reading every order into the program.

Common mistake. Fetching a whole table to filter it in the application. The database can use indexes and send only the matching rows; the program can do neither.